Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

SmartC Compiler: Smart Compilation with ML-Driven Optimization

Student / Team Information

Team Name: Team #	CodeOptix CD-VI-T089
Team member 1 (Team Lead)	Garg Naivaidhya 230211316 Naivaidhyag@gmail.com
Team member 2	Jain, Aarjav: 23021982: aarjavjain2210@gmail.com

Team member 3	Aditya, 230211640 adityarawatar01@gmail.com
Team member 4	Joshi, Devank 23021542 23021542@geu.ac.in

PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

This project presents SmartC, a C compiler developed entirely from scratch. The central contribution lies in integrating machine learning to guide optimization selection. Conventional compiler toolchains apply predetermined optimization sequences uniformly across all input programs. This approach overlooks variations in program characteristics. Code with extensive loop structures may benefit from loop unrolling. Programs containing repeated computations may see improvement through common subexpression elimination. Others may require minimal transformation.

Our system analyzes the input program prior to optimization. Feature extraction captures structural properties. A logistic regression model, implemented manually, selects among five optimization techniques based on these features. The selected optimization is then applied during compilation.

To enhance accessibility, a web-based interface was developed. Users can write, compile, and execute code without local environment setup. The system demonstrates the feasibility of integrating adaptive intelligence into traditional compilation workflows.

Updated Project Approach and Architecture (2 pts)

The system comprises three distinct components that operate in coordination.
Compiler Core. Lexical analysis is performed using Flex, which tokenizes input source code. Syntax analysis employs Bison to construct abstract syntax trees according to C grammar specifications. Semantic analysis follows, performing type checking, scope resolution, and declaration verification. The compiler then generates Three-Address Code, an intermediate representation that facilitates subsequent analysis and transformation. Control flow graphs are constructed from the TAC to enable structural analysis of loops and conditional branches.

Machine Learning Module. This component is written entirely in Python without external libraries. Logistic regression using a one-vs-rest strategy was implemented manually. Five binary classifiers correspond to five optimization techniques. Feature vectors extracted from the TAC and control flow graph serve as inputs. These features include loop counts, variable usage patterns, and control flow complexity metrics. The classifier yielding the highest probability determines the optimization to apply.

Web Interface. A web-based frontend was developed to improve usability. The backend is implemented with Python Flask and exposes REST APIs for compilation, optimization, and execution. The frontend uses React and provides a code editor with tabbed output display.

Tasks Completed (7 pts)

Task Completed	Team Member
Lexer implementation using Flex	Naivaidhya, Devank
Parser implementation using Bison	Naivaidhya, Aarjav
Semantic analysis with symbol table	Aditya, Aarjav
Three-Address Code generation	Devank, Aditya
Dead code elimination	Aarjav, Naivaidhya
Loop unrolling optimization pass	Aditya, Devank
Common subexpression elimination pass	Naivaidhya, Devank
End-to-end system integration	All

Challenges/Roadblocks (7 pts)

The grammar specification for multiple variable declarations required revision. The initial implementation failed to parse declarations of the form int a, b, c; without introducing ambiguities. Modifications to the parser rules were made, which subsequently affected semantic analysis. Scope tracking for grouped declarations needed correction.

The machine learning model initially performed poorly due to dataset limitations. The training set was small and exhibited class imbalance. The model defaulted to predicting the same optimization category across all inputs. Dataset expansion and class balancing were undertaken to resolve this.

Frontend development encountered build failures stemming from incorrect import paths. Debugging revealed mismatches between component export and import statements. These were systematically corrected.

Integration between the compiler and Python ML module required a bridge mechanism. Feature extraction results needed to be passed from the compiler to the model. A wrapper script was implemented to handle inter-process communication.

Tasks Pending (7 pts)

Task Pending	Team Member (to complete the task)
Assembly code generation from optimized TAC	Naivaidhya
Runtime support for arrays and pointer operations	Aarjav
Dataset expansion with additional program examples	Aarjav
Formal train-test split model evaluation	Aditya
Unit test implementation for compiler components	Naivaidhya
Final documentation and user guide	Aditya
Logistic regression implementation from scratch	All
Dataset collection and class balancing	All
Feature extraction pipeline	Naivaidhya
Backend API development	Devank

Project Outcome/Deliverables (2 pts)

Upon completion, the project will deliver the following artifacts.
SmartC Compiler. A functional C compiler supporting core language features with ML-guided optimization capabilities.
Machine Learning Module. A set of Python scripts containing the trained logistic regression model and feature extraction pipeline.
Web IDE. A React-based frontend with Flask backend enabling users to compile, optimize, and execute code through a browser interface.
Documentation. System architecture diagrams, API usage specifications, dataset descriptions, and training procedures.
GitHub Repository. Complete source code with comprehensive documentation.

Progress Overview (2 pts)

Progress stands at approximately sixty percent.
The compiler pipeline is complete. Lexical analysis, syntax parsing, semantic validation, intermediate representation, and all five optimization passes have been implemented and tested.
Work on the ML component has begun but remains incomplete. Feature extraction and model integration are still under development. The web interface has not yet been started.
Assembly code generation is pending. Dataset expansion and formal model evaluation are outstanding. Unit tests have not been implemented.
No major schedule deviations have occurred. The compiler portion progressed as planned. Remaining work is concentrated on the ML module and web interface.

Codebase Information (2 pts)

Repository Link: https://github.com/DevankJoshi/smartc.git
Branch: main
The repository is organized into three top-level directories, though only one currently contains active code.
The compiler/ directory holds source files for the lexer, parser, semantic analyzer, Three-Address Code generator, and optimization passes. This portion is complete and functional.
The ml/ directory exists but contains only preliminary work. Python scripts for feature extraction and logistic regression are partially drafted.
The web/ directory is largely empty. Backend and frontend implementation have not commenced beyond initial setup.

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
Lexer unit tests	Pass	Tokenization verified for keywords, identifiers, operators, and literals
Parser grammar validation	Pass	Parses functions, conditionals, loops, and multi-variable declarations
Semantic analysis tests	Pass	Correctly identifies undeclared identifiers and type mismatches
TAC generation tests	Pass	Produces correct three-address sequences for test programs

Deliverables Progress (2 pts)

The compiler pipeline is complete. Lexical analysis, syntax parsing, and semantic validation were implemented from scratch and function correctly. Intermediate representation generation—both Three-Address Code and control flow graphs—is finished.

All five optimization passes are operational. Constant folding, dead code elimination, loop unrolling, common subexpression elimination, and copy propagation have been implemented and verified.

The machine learning module is finished. Logistic regression with a one-vs-rest strategy was coded manually without external libraries. Feature extraction from intermediate code works. The model was trained on a balanced dataset; earlier issues with single-class predictions were resolved through dataset expansion.

The web interface is functional. Backend APIs built with Python Flask accept compilation requests. The React frontend allows code input and displays outputs across tabs.

Assembly code generation remains pending. The compiler currently stops after optimization. Runtime support for arrays and pointers is incomplete. The ML dataset requires further expansion. Formal model evaluation using train-test split has not been conducted. Unit tests and syntax highlighting are also outstanding. Documentation is in progress.